2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3315-7434-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/COMPSAC65507.2025.00161

Intent Based E2E Automated Test Case Generation for Web Applications Using LLM

1st Zain Ul Abideen
College of Information Science and Technology
Beijing University of Chemical Technology, 100029
Beijing, China
zainrafiq95@gmail.com

2nd Guo Junxia
College of Information Science and Technology
Beijing University of Chemical Technology, 100029
Beijing, China
gjxia@mail.buct.edu.cn

***Abstract*—Ensuring reliability in dynamic web applications is like hitting a moving target, where content constantly changes in response to user interactions and real-time updates. Therefore, testing these applications automatically remains challenging. Additionally, `<canvas>` elements, which render graphical content outside the DOM and lack semantic metadata, makes it difficult to automatically extract and validate their content, leading to incomplete testing in existing methods. This paper proposes a novel approach named E2E-TestGen, which based on intent-driven testing, leverages Large Language Models (LLMs). E2E-TestGen interacts dynamically with the DOM by triggering JavaScript-driven updates and tracking real-time state transitions, ensuring comprehensive validation across all dynamic elements and interactions to improve coverage. For canvas-rendered graphical components, we integrate an image-based object detection and text extraction module to extract meaningful data from visual content, enabling the generation of realistic test case scenarios guided by user intent. The experiments on multiple open-source web applications are conducted, which show E2E-TestGen achieving 88.5% and 80.1% code and branch coverage, respectively, along with a 27.91% improvement in fault detection over baseline approaches.**

***Index Terms*—Automated Testing, Test Generation, Web Testing, Large Language Model**

I. Introduction

Dynamic web applications, powered by frameworks like Angular, Vue.js, and so on, have revolutionized industries with unmatched interactivity and real-time responsiveness [1]. Yet, beneath their seamless interfaces lies a hidden complexity that poses significant challenges for robust testing and validation. JavaScript-driven content, generated dynamically in response to user interactions, poses a significant challenge for traditional automated testing approaches, which predominantly rely on static DOM snapshots. For instance, in SplittyPie [2] (Fig. 1(a)), clicking the "Add Participant" button dynamically adds new input fields to the DOM, altering its structure and influencing subsequent interactions, making the testing more complex. Most model-based approaches [3] [4] [5] depend on static snapshots of the DOM, and face similar challenges as they fail to capture dynamically rendered content. Random-based testing techniques [6] [7] [8] lack directionality, making them ineffective in triggering meaningful workflows. Reinforcement Learning (RL)-based methods [9] [10], even WebQT [11], rely on fixed state abstractions, making it difficult to cover the runtime modifications like dynamically added elements in Fig. 1(b).

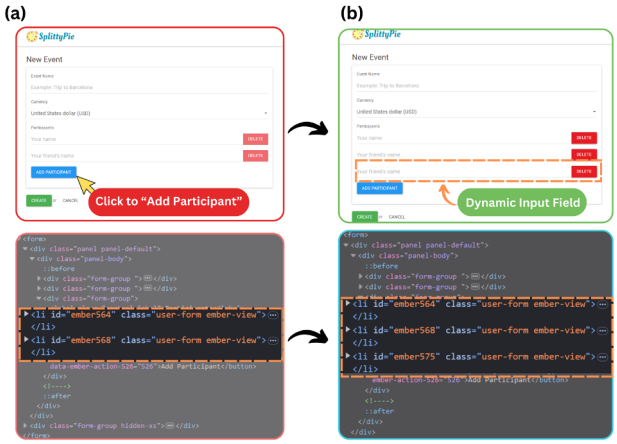

Fig. 1. Illustration of dynamically rendered elements in SplittyPie. (a) Initial state of the form with a single participant input field. (b) New input fields dynamically added to the DOM after clicking 'Add Participant'

In addition, graphical components rendered within `<canvas>` elements introduce another critical challenge for automated testing frameworks. `<canvas>` elements produce pixel-based content that lacks metadata and structural identifiers, making them undetectable by DOM-based inspection methods, preventing existing testing techniques [11] [9] [12] [13] from interacting with related key components rendered within `<canvas>` elements [14]. As a result, workflows involving canvas-based features are frequently untested or rely on manual validation, increasing the likelihood of undetected bugs and regressions [14].

For example, in a Google Maps application [15] (Fig. 2(a)), draggable markers, plotted routes, and map regions are rendered entirely within a `<canvas>` element using JavaScript APIs. Unlike standard DOM elements, these graphical components do not expose structural information, making them inaccessible to automated testing frameworks. Similarly, Fig. 2(b) illustrates a virtual ruler application [16], where measurement markings are dynamically rendered within a `<canvas>`

2836-3795/25/$31.00 ©2025 IEEE
DOI 10.1109/COMPSAC65507.2025.00161

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

element and adjust based on user interactions. Since these elements exist solely as pixel data rather than structured elements, existing model-based and RL-based techniques fail to interact with or validate such workflows, further limiting automated test coverage.

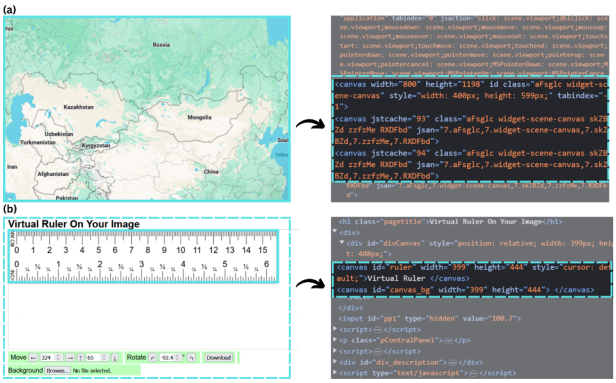

Fig. 2. Examples of <canvas> elements. (a) A dynamic map rendered within a <canvas> element, and (b) a virtual ruler application showcasing <canvas> graphical renderings

To address the limitations of existing automated testing techniques, we propose E2E-TestGen, a framework that combines dynamic DOM exploration, intent analysis, and image-based validation of <canvas> content. E2E-TestGen employs a two-phase DOM exploration process: first, it extracts the static DOM and maps JavaScript event triggers; second, it activates these events, monitors updates, and integrates newly generated nodes to construct a complete and up-to-date DOM representation. For <canvas> elements, an image-based analysis module identifies graphical objects, extracts textual content, and maps their spatial coordinates. The unified representation of DOM and <canvas> content is then processed by LLMs to analyze user intent, generating realistic and comprehensive test case flows. These test flows are transformed into executable test cases to validate complex user interactions and workflows. The primary contributions of this paper are as follows:

1) We introduce an automated method for generating test cases for <canvas> elements, integrating image-based object detection, and text extraction to bridge the gap between DOM-based interactions and pixel-based graphical content.
2) We introduce an "Intent and Test Case Flow" generation module that works with LLM to improve test coverage, edge case detection, and the handling of complex workflows.
3) We evaluated our approach by conducting extensive experiments on multiple open-source web applications, achieving 88.5% and 80.1% code and branch coverage, respectively, along with a 27.91% improvement in fault detection over baseline approaches.

II. Related Work

Reinforcement learning (RL) has been a significant focus in automated testing. RL-based frameworks like WebExplor [9] and WebQT [11] use reward-driven exploration to generate test cases, mimicking human-tester interactions. However, they struggle with critical path identification and handling dynamic web elements due to simplistic reward functions and static state abstractions. Recent advancements, such as DQT [17], integrate graph embeddings to preserve structural and semantic information, but they face scalability challenges in high-dimensional state spaces.

Automated testing has evolved with new strategies designed to expand test coverage and increase the diversity of testing methods. Biagiola et al. [3] introduced a diversity-based method that pre-selects test cases based on their variation from previously generated ones, achieving broader coverage but increasing complexity when handling diverse scenarios. Similarly, FRAGGEN [18] improves test granularity using page fragmentation but adds significant computational overhead. Script-less tools like Cytestion [19] automate test generation based on execution feedback (e.g., HTTP status, console errors), though they remain limited in handling dynamic interactions.

Vision-based techniques have gained traction in addressing challenges in visually dynamic GUIs. UniRLTest [10] integrates vision-based widget detection with reinforcement learning, excelling in diverse state exploration. However, its reliance on accurate widget detection reduces its effectiveness in highly dynamic environments. Similarly, deep learning methods, such as those proposed by Khaliq et al. [20], employ convolutional neural networks (CNNs) for Web UI testing using object detection and NLP-based script generation. While effective in generating functional tests, these approaches require large training datasets and struggle with dynamic web elements.

LLMs have transformed software testing by enabling advanced, context-aware test case generation. VON Similo [21] enhances web element localization with human-like accuracy by selecting top-ranked candidates. Similarly, Mathur et al. [22] combined T5 and GPT-3 to automate test case generation, leveraging T5 for context extraction and GPT-3 for refinement. GPT-3.5-turbo [23] demonstrated comparable code coverage to EvoSuite in Java unit testing, showcasing its potential for automated unit test generation. For Android app testing, ChatGPT-assisted E2E test generation [24] reduced development time by utilizing Gherkin specifications.

Context-aware input generation has also advanced with tools like QTypist [25], which leverages pre-trained LLMs and prompt tuning to produce semantically accurate text inputs for GUI testing. These advancements highlight the growing role of LLMs in automated testing and their versatility across diverse applications, including unit testing, GUI validation, and E2E workflows.

III. E2E-TestGen Approach

E2E-TestGen overcomes the limitations of traditional automated testing by integrating dynamic DOM exploration, image-based canvas validation, and intent-driven test case generation using LLMs. The framework, as illustrated in Fig. 3, comprises three interconnected modules, which are

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

Data Extraction Module, Canvas Analysis Module, and Test Case Generation (TCG) Module. The Data Extraction Module systematically retrieves both static and dynamically generated elements, building a structured dataset for testing. The Canvas Analysis Module processes graphical elements outside the traditional DOM, using image-based object detection, OCR-driven text extraction, and hierarchical structuring to analyze and seamlessly integrate extracted elements finally generating a Hybrid DOM Dataset (HDD). The TCG Module processes the HDD to generate a unified intent for every webpage, capturing the functionalities of all elements and designing realistic test flows that replicate real-world interactions. The Test Case Script Generation (TCSG) process converts these structured flows into executable scripts, validating test cases through iterative feedback. E2E-TestGen seamlessly integrates these modules, enabling robust, automated testing for highly interactive web applications, effectively adapting to runtime modifications.

A. Data Extraction Module

The Data Extraction Module employs a structured crawling mechanism and two-phase DOM exploration process to ensure comprehensive retrieval of both static and runtime elements for automated test generation. Algorithm 1 outlines this process, processing an init file with the web application's configuration and outputting a JSON dataset of extracted static and dynamic elements, attributes, and hierarchical web page links.

The module fetches the web page content using an HTTP GET request and parses it into a structured DOM representation (Lines 1-2). To enable navigation, hyperlinks (`<a>` tags) are identified, their `href` attributes are normalized into absolute URLs, and duplicate links are filtered out to enhance efficiency. A hierarchical navigation structure is then built by extracting linked pages and organizing them within a parent-child tree structure, preserving DOM dependencies (Lines 3-7). For each web page, the first phase extracts key static elements, including interactive components, along with their attributes and maps their parent-child relationships, maintaining the DOM hierarchy (Lines 8-13).

As web applications frequently modify the DOM dynamically, static extraction alone is insufficient. To address this, phase two captures JavaScript-driven modifications by identifying both inline and external scripts. Inline scripts are extracted directly, while external scripts are located through `src` attributes (Lines 14-18). Once identified, these scripts are executed using Selenium WebDriver, triggering JavaScript-driven to load new elements, modify existing ones, and inject additional content into the DOM generated in the first phase. To get those changes that occur within the DOM, we design a Mutation Observer which continuously monitors the DOM during execution, detecting and recording all structural modifications, including element additions, removals, and updates immediately after JavaScript execution (Lines 19-22).

To ensure interaction tracking, event handlers `onclick`, `onchange`, and `onsubmit` are mapped to corresponding DOM elements. Additionally, deep event interception is employed to track dynamically registered event listeners, ensuring that late-bound interactions are also captured (Lines 23-29). The final extracted dataset, stored in JSON format and enriched with structural and behavioral metadata (Lines 30-33), provides a comprehensive representation of the application's dynamic behavior.

B. Canvas Analysis Module

The Canvas Analysis Module converts static canvas-rendered graphics into structured, testable data for automated testing through three core components: Object Detection,

Algorithm 1: Data Extraction Module

Input: `Init_file` (Web application's configuration & target URL)
Output: `dataset` (JSON format containing static and dynamic elements, their attributes, and extracted web page links)

```
1: html_content ← HTTP_GET(target_url)
2: dom ← parse_DOM(html_content)
3: links ← extract_links(dom)
4: for each link in links do
5:    child_dom ← HTTP_GET_and_parse(link)
6:    nav_tree ← add_to_tree(nav_tree, link, parent_page)
7: end for
8: elements ← extract_elements(dom)
9: for each element in elements do
10:    rcrd_relations(element, parent_element)
11:    attr ← extract_attr(element)
12:    dataset ← add_to_dataset(dataset, element, attr)
13: end for
14: script_tags ← find_tags(dom, 'script")
15: for each script in script_tags do
16:    js_files ← js_files ∪ {script} if is_inline(script)
17:    js_files ← js_files ∪ {get_attr(script, 'src")} if not
       is_inline(script)
18: end for
19: for each script in js_files do
20:    execute_script(script), monitor_DOM_updates(dom)
21: end for
22: use_mutation_observer(dom)
23: evnt_hand ← extract_evnt_hand(dom)
24: map_evnt_to_DOM(evnt_hand)
25: for each element in elements do
26:    if has_dynamic_listeners(element) then
27:       capture_event_bindings(element)
28:    end if
29: end for
30: integrate_dynamic_updates(dom)
31: updated_dom ← snapshot_DOM(dom)
32: dataset ← enrich_dataset(dataset, nav_tree, updated_dom)
33: return dataset
```


Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

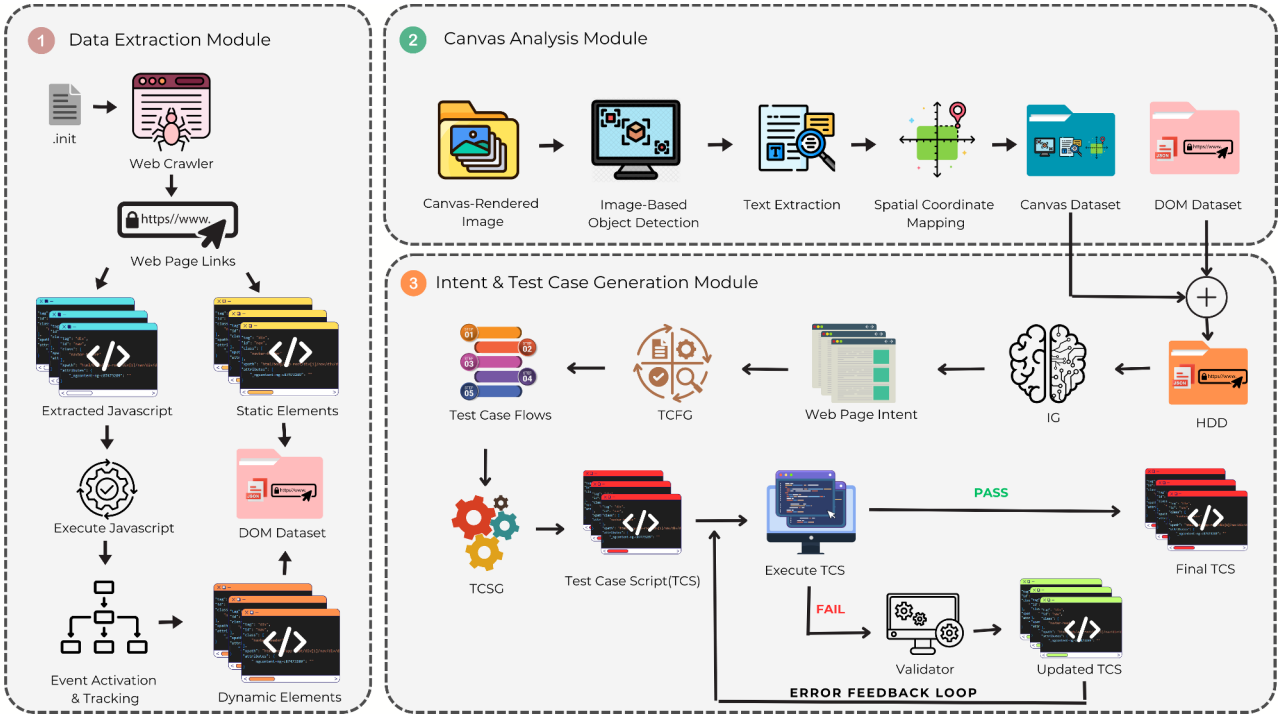

Fig. 3. Overview of E2E-TestGen Framework

Text Extraction, and Hierarchical Structuring. The Object Detection component identifies and classifies UI element, generating a JSON file, which includes object class labels, bounding box coordinates, and confidence scores. The Text Extraction component then extracts textual data from canvas, outputs a JSON file, containing text content along its bounding box coordinates and confidence scores. Finally, the Hierarchical Structuring component combines these files and organizes the objects and text based on their relationships, linking text to its corresponding UI elements to ensure they are connected in a meaningful way. The result is a unified HDD.

a) Object Detection: Identifying and classifying elements within a canvas-rendered UI is the first step in making them accessible for automated testing. Extracting meaningful components requires more than pixel analysis. To achieve this, we need precise object recognition to distinguish graphical elements from functional UI components and construct a structured representation to align with the normal UI description. The object detection process utilizes YOLOv8, a deep learning model fine-tuned on the RICO dataset, which categorizes UI components into 25 distinct object classes. The model was trained with a batch size of 16, an initial learning rate of 0.001, and the Adam optimizer over 47 epochs to balance generalization and overfitting. The model generates bounding boxes, class labels, and confidence scores for each detected object. To enhance accuracy and reduce noise, detections with confidence scores below 0.5 are discarded. Additionally, Soft-Non-Maximum Suppression (Soft-NMS) is applied to resolve overlapping predictions, ensuring that only the most confident bounding box is retained for each object. The final extracted data is stored in the JSON file, which forms the foundation for further analysis. Since the RICO dataset is primarily designed for Android UIs, for adapting to the web UI, we extend the process with text extraction and hierarchical structuring of canvas-extracted objects and texts. The HDD generated through this process is then refined by the IG Module, which corrects object misclassifications using contextual understanding. This iterative refinement ensures a more precise object detection.

b) Text Extraction: The textual data together with graphical structures of canvas-rendered elements can also provide useful information. To accomplish this, we employed PaddlePaddle OCR (PP-OCR). Preprocessing refines bounding boxes from object detection using bilateral filtering and Contrast-Limited Adaptive Histogram Equalization (CLAHE) to enhance visibility and reduce noise. Misaligned text is aligned with Hough Transform-based deskewing, while Bezier curve fitting with geometric transformations is used to correct curved and distorted text for accurate OCR. The preprocessed image undergoes text extraction through PP-OCR, which integrates DBNet for precise text region segmentation and CRNN for sequence-based recognition, ensuring robust performance across diverse styles and layouts. The output of this process is the JSON file containing text extraction data.

c) Hierarchical Structuring of Object and Text Data: To effectively utilize the extracted canvas data, it is essential to establish meaningful relationships between detected objects

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PROMPT STRUCTURE FOR TEST CASE GENERATION, TEST CASE FLOW GENERATION, AND TEST CASE SCRIPT GENERATION

Template Component	Intent Generation (IG)	Test Case Flow Generation (TCFG)	Test Case Script Generation (TCSG)
Role Assignment (RA)	You are an AI-powered system designed for automated software testing. Your task is to analyze the Hybrid DOM Dataset (HDD) JSON file and generate functional and contextual intents for all webpage elements, including both static and dynamic components.	You are an AI-powered system specialized in workflow automation. Your task is to construct step-by-step user interaction flows based on the extracted intents, ensuring full interaction coverage.	You are an AI-driven test automation system specialized in Selenium WebDriver. Your task is to convert test case flows into robust, modular scripts optimized for execution.
Input Specification (IS)	**Input:** JSON file containing HDD data, including element attributes, hierarchy, and JavaScript interactions.	**Input:** JSON file containing extracted intents, describing functional roles and contextual relationships of webpage elements.	**Input:** JSON file containing test case flows, including user interactions, DOM JSON (XPath references), and application setup instructions.
Processing Instructions (PI)	Generate structured intent descriptions for each element's purpose and behavior, while maintaining the DOM hierarchy. Identify interactive elements, including dynamic components and JavaScript interactions.	Generate test workflows covering user interactions such as navigation, form submissions, and dynamic behaviors (e.g., AJAX requests). Handle edge cases (e.g., invalid inputs, server errors) and synchronize UI updates dynamically.	Convert test workflows into Selenium WebDriver commands, mapping interactions using XPath selectors. Implement setup steps (e.g., login, cookies) and use dynamic waits for asynchronous events. Ensure scripts are modular and optimized for execution.
Output Specification (OS)	JSON output containing: - Element Type (e.g., Button, Input Field) - Attributes (e.g., id, class) - Functional and contextual intents - JavaScript interactions (e.g., onclick, onsubmit)	JSON output containing: - Workflow Name (e.g., User Registration) - Steps: User Actions and Expected Results - DOM element references (e.g., id, XPath)	Selenium test scripts containing: - Preconditions (e.g., login setup) - Steps: User Actions and Validations - Error Handling for edge cases - Dynamic waits for AJAX and loading states

and textual data. This process organizes the data into a well-defined hierarchical structure, improving the accuracy and efficiency of the testing process. Spatial proximity algorithms are applied to link detected text to its corresponding UI components, ensuring a structured representation of the canvas content. If text elements are fully contained within the bounds of a UI component, they are considered children of that object. For text outside of these bounds but within a defined proximity, a distance-based algorithm links it to the nearest component, while un-associated text is classified as standalone at the root level.

The resulting hierarchy is stored as a tree-like structure, where each object node contains child text nodes, ensuring all relationships are preserved. This data is then integrated into the DOM by linking enriched canvas elements to their corresponding DOM nodes, identified by `id` and `class`, within the pre-existing DOM dataset extracted during the Data Extraction Module. The updated DOM dataset, called the Hybrid DOM Dataset (HDD), represents a unified structure that encapsulates both static and dynamically extracted elements, providing a comprehensive dataset for test case generation.

C. Test Case Generation (TCG) Module

The TCG module is designed to ensure comprehensive and efficient test coverage by utilizing three interconnected processes: Intent Generation (IG), Test Case Flow Generation (TCFG), and Test Case Script Generation (TCSG), all powered by LLM. The prompt structure for each of these processes is provided in Table I.

a) Intent Generation (IG): The IG process analyzes the HDD to generate a unified intent representation for the webpage, capturing the functional and contextual roles of its elements. It produces a structured JSON output detailing each element's type, attributes, functionality, hierarchical role, and interactive behavior, all aligned with the DOM. This output serves as the foundation for generating TCFs.

b) Test Case Flow Generation (TCFG): The TCFG process converts structured intents from the IG process into detailed TCFs that validate various UI interactions and behaviors. These flows are designed to cover a wide range of scenarios, including element functionality, error handling, dynamic interactions, and multi-step workflows. The output captures preconditions, user actions, expected validations, and outcome assertions for each flow, which serves as the foundation for generating TCSs.

c) Test Case Script Generation (TCSG) : TCSG process converts the TCFs into executable Selenium WebDriver scripts. These scripts undergo an execution phase to validate their correctness. If successful, they are deployed to the automated testing pipeline. If an error occurs, the Error Feedback Loop (EFL) is triggered. Validator reviews the logs, identifies issues, and generates updated scripts for re-validation. This process repeats up to three times, and if the script passes, it is deployed; if it fails, the most recent version is retained as the final script.

IV. EVALUATION

To demonstrate the effectiveness of our intent-based E2E automated test case generation approach, we conducted a series of experiments to answer the following research questions:

- **RQ1:** How effective is E2E-TestGen in handling dynamic elements and achieving statement and branch coverage?

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

- **RQ2:** Are all modules designed in E2E-TestGen effective?
- **RQ3:** Do test cases generated through E2E-TestGen's structured intent and test flow generation process improve coverage compared to the direct LLM-based test case generation method?
- **RQ4:** How effective are the test cases generated by E2E-TestGen in detecting faults?

A. Experimental Setup

Dataset: For RQ1 and RQ2, we utilized a benchmark dataset of seven popular open-source web applications, which are extensively referenced in previous studies [11] [9] [3]. The details of those applications are listed in Table II. All of them received over 100 stars on GitHub, indicating strong community adoption.

In addition, to evaluate the effectiveness of intent and test flow generation in our approach, in RQ3, we compare with the baseline approach TestPilot [26], which generates unit tests directly from LLM. Aligned with the dataset used in baseline approach, we conduct experiments on 25 npm packages, each with diverse characteristics. The details of these packages are listed in Table III.

To verify the bug detection ability of the test cases generated with our approach, in RQ4, we use four well-known open-source web applications listed in Table IV according to a baseline approach. Since the baseline is not open source, using this dataset ensures a relatively fair comparison.

Configuration: We employed GPT-3.5 Turbo as the LLM in our framework due to its widespread adoption and alignment with the baseline configuration. To ensure a fair and consistent evaluation, all experiments were repeated five times using the same setup as the baseline.

TABLE II
BENCHMARK DATASET FOR RQ1 & RQ2

Web Application	KLOC	Framework	Stars
Petclinic	25.7	Angular.js	197
Splittypie	27.9	Ember.js	172
Phoenix	34.6	Phoenix/React	2.5k
Timeoff	40.8	Node.js	920
Dimeshift	33.1	Backbone.js	193
Retroboard	29.8	React	773
Hospital	19.4	Php	428

TABLE III
BENCHMARK DATASET FOR RQ3: NPM PACKAGES AND THEIR CHARACTERISTICS

Package	LOC	Tests	Downloads	Package	LOC	Tests	Downloads
glob	314	22	103M	plural	53	24	9.7K
fs-extra	822	417	79M	dirty	76	10	18K
graceful-fs	208	11	48M	geo-point	31	3	417
jsonfile	46	43	48M	uneval	32	12	23K
bluebird	3.1K	238	26M	image-downloader	100	185	5
q	736	214	14M	crawler-url-parser	205	14	184
rsvp	565	171	8.6M	gitlab-js	136	16	1
memfs	2.2K	265	13M	core	1.6K	420	1
node-dir	244	55	6M	omnitool	1.6K	420	1
zip-a-folder	25	5	95K	countries-and-timezones	913	307	115K
js-sdsl	1.5K	88	9.7M	simple-statistics	917	307	103K
quill-delta	395	180	1.6M	pull-stream	78	14	78K
complex.js	393	321	497K				

TABLE IV
BENCHMARK DATASET FOR RQ4: INJECTED FAULTS AND ACTIONABLE ELEMENTS

Project	KLOC	Actionable Elements	Injected Faults
Petclinic	25.7	32	45
Bistro restaurant	33.4	191	40
Learn educational	19	221	45
School educational	30.2	208	35

Baseline Approaches: To assess the effectiveness of E2E-TestGen, we compared it against state-of-the-art baseline approaches, each aligned with a specific research question.

- **WebExplor [11]:** It employs curiosity-driven reinforcement learning to create testing action sequences and builds an automaton for guidance during the testing process.
- **WebQT [9]:** It utilizes reinforcement learning with an innovative reward model to mimic human testers interactions and a state abstraction technique to reduce redundancy.
- **TestPilot [26]:** It is an LLM-based test generation tool for JavaScript that automatically generates unit tests for the methods in a given project's API.
- **Testar [27]:** It is a scriptless tool for automated test generation. It uses intelligent agents to generate test sequences by transitioning the system through states, detecting faults, and identifying violations in functionality, accessibility, and security.
- **Cytestion [19]:** It is a script-less GUI testing tool that explores web applications to detect visible faults that causes failures such as crashes, errors and unexpected behaviors.

B. How effective is E2E-TestGen in handling dynamic elements and achieving statement and branch coverage? (RQ1)

To evaluate the effectiveness of E2E-TestGen in handling dynamic elements and achieving comprehensive coverage, we measured statement and branch coverage using nyc [28], which is also used by baseline approaches. As shown in Table V, E2E-TestGen consistently outperforms baseline approaches, achieving an average statement coverage of 88.5% and branch coverage of 80.1%, compared to 30.4%/13.59% for WebExplor and 71.47%/52.87% for WebQT.

TABLE V
STATEMENT AND BRANCH COVERAGE ACROSS WEB APPLICATIONS

Web Application	Statement Coverage			Branch Coverage		
	WebExplor	WebQT	E2E-TestGen	WebExplor	WebQT	E2E-TestGen
PetClinic	41.57%	95.83%	97.3%	0%	85%	91.4%
Splittypie	32.95%	64.67%	91.7%	7.69%	45.16%	79.8%
Phoenix	24.25%	79.10%	87.2%	22.37%	69.74%	78.3%
Retroboard	58.19%	83.82%	89.4%	22.81%	60.23%	82.8%
Dimeshift	18.94%	46.31%	79.8%	14.72%	39.17%	71.5%
Timeoff	6.54%	48.27%	84.5%	13.98%	39.87%	76.1%
Hospital	-	82.35%	89.8%	-	30.97%	81.3%
Average	**30.40%**	**71.47%**	**88.5%**	**13.59%**	**52.87%**	**80.1%**

The effectiveness of E2E-TestGen is particularly evident in web applications like DimeShift and TimeOff, which contain a significant number of dynamic elements. For DimeShift, E2E-TestGen achieves 79.8% statement coverage and 71.5% branch

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

coverage, compared to WebQT's 46.3% and 39.1%. Similarly, for TimeOff, E2E-TestGen achieves 84.5% statement coverage and 76.1% branch coverage, while WebQT records only 48.2% and 39.8%, respectively.

The baseline approaches cannot detect elements that emerge due to user interactions, leaving key UI paths untested in JavaScript-driven applications. In contrast, E2E-TestGen dynamically explores the application, continuously adapting to changes in the DOM during execution. This enables it to interact with run-time elements in real time, ensuring enhanced coverage of asynchronous workflows and dynamically rendered UI components, where baseline methods fall short.

Answer to RQ1 :

E2E-TestGen consistently outperforms baseline approaches in achieving higher statement and branch coverage, with averages of 88.5% and 80.1%, respectively. Its ability to handle dynamic elements in JavaScript-driven applications like DimeShift and TimeOff highlights its effectiveness in covering UI paths that baseline methods fail to test.

C. Are all modules designed in E2E-TestGen effective? (RQ2)

To evaluate the contribution of individual modules within E2E-TestGen, we conducted an ablation study, isolating the performance of each module. Four ablations were tested: *E2E-TestGen*$_{NoJS}$, excluding JavaScript extraction; *E2E-TestGen*$_{NoIF}$, excluding both IG and TCFG; *E2E-TestGen*$_{NoF}$, excluding TCFG; and *E2E-TestGen*$_{NoI}$, excluding IG. Results shown in Table VI and Table VII, the statement and branch coverage of each situation.

TABLE VI
ABLATION STUDY: STATEMENT COVERAGE ACROSS WEB APPLICATIONS

Web Application	E2E-TestGen$_{NoJS}$	E2E-TestGen$_{NoIF}$	E2E-TestGen$_{NoF}$	E2E-TestGen$_{NoI}$
PetClinic	58.2%	65.1%	83.7%	91.0%
SplittyPie	51.4%	60.2%	78.6%	86.3%
Phoenix	40.5%	55.8%	72.4%	81.1%
RetroBoard	49.3%	63.5%	81.2%	85.9%
DimeShift	34.8%	52.9%	67.3%	76.1%
TimeOff	31.6%	48.5%	69.8%	78.5%
Hospital	28.7%	50.3%	71.4%	79.2%
Average	**42.1%**	**56.6%**	**74.9%**	**82.6%**

TABLE VII
ABLATION STUDY: BRANCH COVERAGE ACROSS WEB APPLICATIONS

Web Application	E2E-TestGen$_{NoJS}$	E2E-TestGen$_{NoIF}$	E2E-TestGen$_{NoF}$	E2E-TestGen$_{NoI}$
PetClinic	28.3%	43.8%	67.9%	81.2%
SplittyPie	23.6%	38.4%	61.5%	77.4%
Phoenix	18.2%	33.7%	56.2%	74.3%
RetroBoard	21.7%	39.2%	64.7%	80.1%
DimeShift	16.5%	31.8%	54.1%	68.7%
TimeOff	15.8%	30.5%	57.2%	71.3%
Hospital	19.1%	35.2%	58.3%	70.6%
Average	**20.4%**	**36.1%**	**60.0%**	**74.8%**

E2E-TestGen$_{NoJS}$ performs the worst (42.1% statement, 20.4% branch coverage), as JavaScript extraction is foundational for identifying runtime dynamic elements. *E2E-TestGen*$_{NoIF}$ achieves moderate coverage (56.6% statement, 36.1% branch) as JavaScript elements are extracted, allowing interaction with dynamic components. However, without IG and TCFG, the tests lack proper sequencing and context, which results in missed interactions between UI elements and incomplete test scenarios. *E2E-TestGen*$_{NoF}$ performs better (74.9% statement, 60.0% branch coverage), with IG analyzing the functional roles of all UI elements, providing a clear understanding of their purpose within the application. However, without TCFG, the interactions are not organized in a logical sequence, leaving gaps in coverage. *E2E-TestGen*$_{NoI}$ achieves the highest coverage among ablations (82.6% statement, 74.8% branch) by leveraging TCFG to structure interactions into logical sequences, ensuring enhanced coverage.

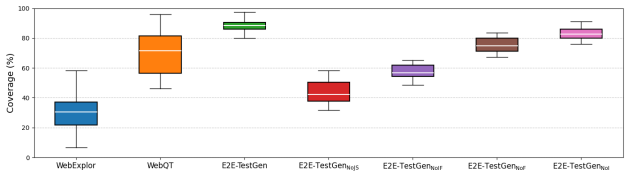

Fig. 4. Statement coverage comparison of WebExplor, WebQT, E2E-TestGen, and its variations.

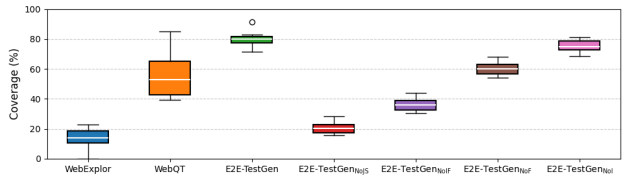

Fig. 5. Branch coverage comparison of WebExplor, WebQT, E2E-TestGen, and its variations.

Figures 4 and 5 show that E2E-TestGen outperforms baseline approaches and its variations in both statement and branch coverage. This is due to its use of both IG and TCFG for test case generation. IG identifies the functional and contextual roles of UI elements, ensuring that user interactions are accurately captured based on their role in the application. These interactions are then organized by TCFG into structured workflows, enabling a more comprehensive exploration of both static and dynamic elements, making E2E-TestGen the most effective method for achieving optimal test coverage.

Answer to RQ2 :

The ablation study shows that all modules in E2E-TestGen are essential for achieving optimal test coverage. Excluding key components like JavaScript extraction, IG and TCFG significantly reduces coverage, highlights their essential role in capturing dynamic elements.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

D. Do test cases generated through E2E-TestGen's structured intent and test flow generation process improve coverage compared to the direct LLM-based test case generation method?(RQ3)

To evaluate whether intent-based test case generation is more effective than direct LLM-based test case generation, we extend TestPilot [26], an opensource framework published in 2024, by incorporating our two key components: *IG* and *TCFG*. This results in the creation of IntentPilot, a new framework that uses intent-based analysis to generate test cases.

TABLE VIII
RESULT COMPARISON OF TESTPILOT AND INTENTPILOT.

Project	TestPilot				IntentPilot			
	Tests	Pass (%)	Stmt (%)	Branch (%)	Tests	Pass (%)	Stmt (%)	Branch (%)
glob	68	18 (26.5)	71.3	66.3	88	46 (52.2)	75.2	68.7
fs-extra	471	277 (58.8)	58.8	38.9	512	310 (60.5)	62.6	42.1
graceful-fs	345	177 (51.4)	49.3	33.3	394	210 (53.0)	56.0	35.0
jsonfile	13	6 (46.0)	38.3	29.4	21	10 (47.6)	45.0	35.2
bluebird	370	204 (55.2)	50.0	50.0	413	230 (55.6)	55.3	52.7
q	323	162 (50.2)	70.4	53.7	376	215 (57.2)	74.1	56.0
rsvp	109	67 (64.2)	70.1	55.1	127	85 (66.9)	79.8	58.3
memfs	1037	471 (45.4)	81.1	55.8	1090	580 (53.2)	86.0	60.0
node-dir	40	19 (47.1)	64.1	50.8	54	27 (50.0)	66.7	56.1
zip-a-folder	14	10 (48.1)	73.3	53.8	19	9 (50.0)	70.7	56.1
js-sdsl	409	46 (54.5)	39.3	24.3	456	60 (52.6)	37.5	37.6
quill-delta	152	11 (7.3)	70.2	50.0	181	60 (33.2)	72.5	53.3
complex.js	209	121 (58.0)	70.6	46.5	255	150 (60.5)	75.0	50.2
pull-stream	83	34 (41.0)	70.7	52.6	100	50 (50.0)	73.5	55.0
countries-and-timezones	28	13 (46.4)	93.1	69.1	38	20 (57.1)	95.0	72.0
simple-statistics	353	250 (70.9)	87.8	71.3	387	300 (77.5)	90.0	75.5
plural	13	9 (61.5)	74.5	59.1	22	17 (77.5)	75.0	62.0
dirty	10	6 (45.3)	87.5	65.4	19	14 (66.7)	90.0	68.0
geo-point	76	50 (65.8)	78.5	65.8	92	67 (72.2)	80.0	68.5
uneval	7	2 (28.6)	68.6	58.3	13	7 (40.0)	74.0	61.6
image-downloader	5	4 (80.0)	63.6	50.0	12	11 (85.7)	68.0	55.0
crawler-url-parser	14	2 (14.3)	51.4	46.4	26	15 (57.0)	58.0	50.0
gitlab-js	141	14 (9.9)	51.4	50.0	178	53 (29.7)	69.0	56.5
core	85	13 (15.3)	78.3	55.2	123	78 (63.4)	82.0	62.0
omnitool	1033	330 (31.9)	74.2	55.2	1107	734 (66.3)	77.0	68.5
AVERAGE	**-**	**48.0**	**70.2**	**52.8**	**-**	**62.0**	**79.2**	**63.4**

The results in Table VIII show significant improvements in test coverage with IntentPilot. While the baseline TestPilot [26] achieves an average statement coverage of 70.2% and branch coverage of 52.8%, IntentPilot demonstrates a significant improvement, reaching 79.2% statement coverage and 63.4% branch coverage, reflecting gains of 9.0% and 10.6%, respectively. Notable improvements are observed in projects like *core* (statement: 78.3% to 82.0%, branch: 55.2% to 62.0%) and *omnitool* (statement: 74.2% to 77.0%, branch: 55.2% to 68.5%). In addition to improving coverage metrics, IntentPilot significantly reduces error rates in generated tests compared to the baseline. Timeout errors, primarily due to missing `done()` calls, decrease from 22.7% to 11.8% with the test case flow module ensuring proper handling of asynchronous workflows. Correctness errors caused by type mismatches drop from 20.0% to 9.3%, while assertion errors fell from 19.2% to 8.5% as IntentPilot aligned test cases with expected input-output relationships, producing robust, behavior-validating test cases.

Figure 6 shows that the test cases generated by the baseline approach for the glob package encounter timeout errors due to improper asynchronous handling. IntentPilot resolves this by correctly using the `done()` call. Similarly, Figure 7 shows how IntentPilot enhances test coverage for the mean function in the simple-statistics package by addressing assertion errors through type validation and edge case tests.

Answer to RQ3 :

IntentPilot significantly improves test coverage compared to TestPilot, achieving gains of 9.0% in statement coverage and 10.6% in branch coverage, while reducing timeout, correctness, and assertion errors, demonstrating its effectiveness in generating comprehensive test cases.

TestPilot

```
const glob = require('glob');
const expect = require('chai').expect;

describe('Glob Pattern Test Without TCF',
function() {
it('should find all .js files in the src directory',
function(done) {
glob("**/*.js", { cwd: './src' }, function(err, files) {
if (err) done(err); // Proper error handling
ensures the test doesn't hang.

expect(files).to.be.an('array').that.is.not.empty;
// Missing done() here will cause Mocha to wait
indefinitely and result in a timeout error.
});
});
});
```

IntentPilot

```
const glob = require('glob');
const expect = require('chai').expect;

describe('Glob Pattern Test With Test Case
Flow', function() {
it('should find all .js files in the src directory',
function(done) {

glob("**/*.js", { cwd: './src' }, function(err, files) {
if (err) return done(err);

// Validates the file array
expect(files).to.be.an('array').that.is.not.empty;

done(); // Ensures that Mocha doesn't time out
});
});
});
```

Fig. 6. Comparison of TestPilot and IntentPilot, showing how IntentPilot resolves timeout errors with proper asynchronous handling in glob package.

TestPilot

```
const ss = require('simple-statistics');
const expect = require('chai').expect;

describe('Simple Statistics Mean Test Without
TCF', function() {
it('should calculate the correct mean of an
array', function(done) {
const result = ss.mean([1, 2, 3, 4, 5]);

// Incorrect assertion: The function actually
returns a number, but the test expects a string
expect(result).to.equal('3'); // This failed
because mean([1, 2, 3, 4, 5]) is a number, not a
string

});
});
```

IntentPilot

```
const ss = require('simple-statistics');
const expect = require('chai').expect;

describe('Simple Statistics Mean Test With Test Case Flow',
function() {
it('should calculate the correct mean of an array',
function(done) {
const result = ss.mean([1, 2, 3, 4, 5]);

// Assert that the result is a number
expect(result).to.be.a('number');

// Assert that the mean is correct
expect(result).to.equal(3); // Corrected assertion: mean is a
number, not a string

// Edge Case 1: Test with an empty array
expect(ss.mean([])).to.be.NaN; // Mean of an empty array
should be NaN

// Edge Case 2: Test with an array of negative and positive
numbers
expect(ss.mean([-1, 1])).to.equal(0);

done();
});
});
```

Fig. 7. Comparison of TestPilot and IntentPilot implementations for testing the mean function in the simple-statistics package, highlighting how IntentPilot improves assertion correctness and handles edge cases, such as empty arrays

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

E. How effective are the test cases generated by E2E-TestGen in detecting faults? (RQ4)

To evaluate E2E-TestGen's effectiveness in detecting failures, we use a faults dataset created by the baseline approach Cytestion [19], which contains 165 faults across four web applications. The dataset only provides fault descriptions without the integrated applications. We inject those faults into the applications [1] to enable a fair evaluation when compared with the baseline approach. The faults include data-layer issues (e.g., schema mismatches, primary key changes), controller-level errors (e.g., condition flipping, null values, route mapping issues), validation faults (e.g., altered input validations, incorrect field mappings), front-end issues (e.g., typographical errors, broken bindings), and routing faults (e.g., truncated or incorrect endpoints).

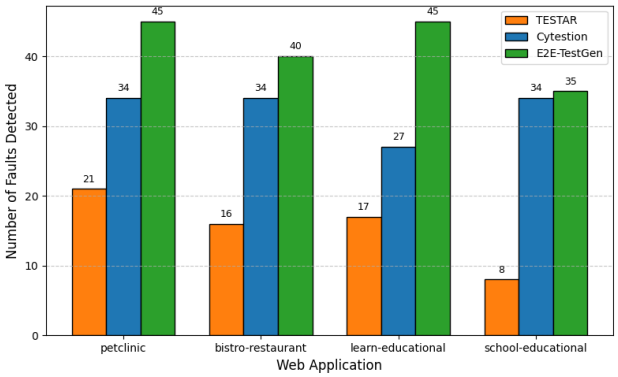

Fig. 8. Fault detection comparison of TESTAR, Cytestion, and E2E-TestGen across web applications.

The results in Fig 8 show the fault detection performance across four web applications. In Petclinic, out of 45 injected faults, Testar detects 21, and Cytestion detects 34. In Bistro Restaurant, out of 40 injected faults, Testar detects 16, and Cytestion detects 34. For Learn Educational, out of 45 injected faults, Testar detects 17, and Cytestion detects 27. In School Educational, out of 35 injected faults, Testar detects 8, and Cytestion detects 34.

Testar detects only a limited number of faults, ranging from 8 in the school-educational application to 21 in the petclinic application. It seems weak at navigating deep action sequences. Cytestion detects more faults than Testar but struggles with dynamic workflows and asynchronous processes. Its static wait times and synchronization issues often lead to missed faults.

In contrast, E2E-TestGen detected all faults for all test objects, demonstrating its ability to handle both static and dynamic elements while exploring deep application states.

[1] https://github.com/E2E-TestGen/Web-Applications.git

Answer to RQ4 :

E2E-TestGen effectively detects faults, outperforming both Testar and Cytestion. Unlike baseline approaches, which struggle with dynamic workflows, E2E-TestGen identifies all injected faults, ensuring comprehensive detection across various application states.

V. Threats To Validity

Dynamic Content Handling: Managing dynamic elements in web applications is challenging for automated testing. Our approach uses a two-phase DOM exploration to capture state and behavior accurately. By integrating intent analysis, it identifies user-driven interactions, while structured test case flows ensure comprehensive validation of dynamic content, leading to effective test case generation.

Generalizability: We implemented out approach on seven open-source web applications which are widely used in existing works [11] [9] [3]. The evaluation shows its robustness across different architectures and frameworks. However, to further ensure generalizability our future work will include testing on broader range of web applications.

Scalability: The approach may face challenges when dealing with very large and complex web applications with numerous web pages and dynamic elements. We mitigated this threat by using an advanced web crawler and processing algorithm to handle large datasets efficiently and employing distributed computing techniques to parallelize the crawling and testing process.

Algorithm Bias: Traditional models that rely on specific dataset might introduce biases, affecting the accuracy and reliability of the generated test cases. We mitigated this by using LLM, which is trained on a vast and diverse dataset, ensuring a more balanced and fair output.

Experimental Setup: Randomness can pose a threat to the validity of our evaluation. To mitigate this we ran our approach for 5 times and used the average of results, ensuring consistent and reliable results.

VI. Conclusion

In this paper, we propose an automated intent-base E2E test case generation approach for Web Applications, named E2E-TestGen, which focuses on addressing key challenges in handling dynamic content and canvas-rendered elements. By integrating intent analysis cooperating with test flow generation together with LLMs, our framework generates executable test cases and achieves higher statement and branch coverage than existing approaches. Furthermore, E2E-TestGen employs an image-based module to analyze canvas-rendered graphical content, bridging the gap between pixel-based and DOM-based testing. The framework is evaluated through extensive experiments, where it outperforms state-of-the-art testing methods, achieving 88.5% statement coverage and 80.1% branch coverage, surpassing baseline approaches. Ablation study highlights the role of JavaScript extraction, intent analysis, and structured test flows in enhancing test coverage. IntentPilot,

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.

an enhanced version of TestPilot, improves test coverage, increasing statement coverage by 9.0% (from 70.2% to 79.2%) and branch coverage by 10.6% (from 52.8% to 63.4%), while also reducing error rates. In addition to improving coverage, E2E-TestGen excels in fault detection, achieving a 27.91% improvement over baseline methods. These results show the robustness of E2E-TestGen in exploring complex workflows, identifying edge cases, and improving fault detection in web applications.

REFERENCES

[1] W3Techs, "Historical trends in the usage of client-side languages for websites," 2023, [Online]. Available: https://w3techs.com/technologies/historyoverview/clientsidelanguage/all/Y.

[2] S. Team, "Splittypie: Dynamic web application for testing," https://splittypie.com/, 2025.

[3] M. Biagiola, A. Stocco, F. Ricca, and P. Tonella, "Diversity-based web test generation," in *European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 142–153.

[4] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu, and Z. Su, "Guided, stochastic model-based gui testing of android apps," in *Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 245–256.

[5] K. Mao, M. Harman, and Y. Jia, "Sapienz: Multi-objective automated testing for android applications," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, 2016, pp. 94–105.

[6] Google, "Monkey - android testing tools," 2024, [Online]. Available: https://developer.android.com/studio/test/other-testing-tools/monkey.

[7] Crawljax, "Crawljax," 2024, [Online]. Available: https://github.com/crawljax/crawljax.

[8] A. Machiry, R. Tahiliani, and M. Naik, "Dynodroid: An input generation system for android apps," in *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, 2013, pp. 224–234.

[9] Y. Zheng, Y. Liu, X. Xie, Y. Liu, L. Ma, J. Hao, and Y. Liu, "Automatic web testing using curiosity-driven reinforcement learning," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2021, pp. 423–435.

[10] Z. Zhang, Y. Liu, S. Yu, X. Li, Y. Yun, C. Fang, and Z. Chen, "Unirltest: Universal platform-independent testing with reinforcement learning via image understanding," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2022, pp. 805–808.

[11] X. Chang, Z. Liang, Y. Zhang, L. Cui, Z. Long, G. Wu, Y. Gao, W. Chen, J. Wei, and T. Huang, "A reinforcement learning approach to generating test cases for web applications," in *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*, 2023, pp. 13–23.

[12] M. Pan, A. Huang, G. Wang, T. Zhang, and X. Li, "Reinforcement learning based curiosity-driven testing of android applications," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 153–164.

[13] Y. Zheng, X. Xie, T. Su, L. Ma, J. Hao, Z. Meng, Y. Liu, R. Shen, Y. Chen, and C. Fan, "Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning," in *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering*, 2019, pp. 772–784.

[14] A. Soetens, J. F. Bastos, and G. Franch, "Web canvas testing through visual inference," in *Proceedings of the IEEE International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, Porto, Portugal, 2020, pp. 126–133.

[15] G. Maps, "Google maps," 2024, [Online]. Available: https://maps.google.com.

[16] "Virtual ruler on your image," 2024, [Online]. Available: https://www.ginifab.com/feeds/cmtoinch/virtual_ruler_on_your_Image.h tml.

[17] Y. Lan, Y. Lu, Z. Li, M. Pan, W. Yang, T. Zhang, and X. Li, "Deeply reinforcing android gui testing with deep reinforcement learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. New York, NY, USA: Association for Computing Machinery, 2024.

[18] R. K. Yandrapally and A. Mesbah, "Fragment-based test generation for web apps," *IEEE Transactions on Software Engineering*, vol. 49, pp. 1086–1101, March 2023.

[19] T. S. d. Moura, E. L. G. Alves, H. F. d. Figueiredo, and C. d. S. Baptista, "Cytestion: Automated gui testing for web applications," in *Proceedings of the XXXVII Brazilian Symposium on Software Engineering (SBES '23)*. New York, NY, USA: Association for Computing Machinery, 2023, pp. 388–397.

[20] Z. Khaliq, D. A. Khan, and S. U. Farooq, "Using deep learning for selenium web ui functional tests: A case study with e-commerce applications," *Engineering Applications of Artificial Intelligence*, vol. 117, January 2023.

[21] M. Nass, E. Alegroth, and R. Feldt, "Improving web element localization by using a large language model," *CoRR*, vol. abs/2310.02046, 2023.

[22] A. M. et al., "Automated test case generation using t5 and gpt-3," in *2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS)*. IEEE, 2023, pp. 1986–1992.

[23] V. H. Guilherme and A. M. R. Vincenzi, "An initial investigation of chatgpt unit test generation capability," in *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing (SAST 2023)*, Campo Grande, MS, Brazil, 2023.

[24] B. García, M. Leotta, F. Ricca, and J. Whitehead, "Use of chatgpt as an assistant in the end-to-end test script generation for android apps," in *Proceedings of the 15th ACM International Workshop on Automating Test Case Design, Selection, and Evaluation (A-TEST '24)*, Vienna, Austria, 2024.

[25] Z. L. et al., "Fill in the blank: Context-aware automated text input generation for mobile gui testing," in *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Melbourne, Australia, 2023, pp. 1355–1365.

[26] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.

[27] T. E. Vos, P. Aho, F. P. Ricos, O. Rodriguez-Valdes, and A. Mulders, "Testar–scriptless testing through graphical user interface," *Software Testing, Verification and Reliability*, vol. 31, no. 3, p. e1771, 2022.

[28] IstanbulJS, "Nyc - the istanbul command line interface," 2024. [Online]. Available: https://github.com/istanbuljs/nyc

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:55:18 UTC from IEEE Xplore. Restrictions apply.